Curso
Tendencias Aplicadas de Ingenería de Software

CI/CD

¿Qué es Continuous Integration?

- La Integración Continua (Continuous Integration o CI) es una práctica de ingeniería de software que propone que todos los desarrolladores integren frecuentemente sus cambios en un repositorio compartido. La idea central es evitar los problemas asociados a las integraciones tardías, donde los conflictos entre módulos, dependencias y versiones pueden acumularse durante días o semanas.
- Antes de la adopción masiva de CI, era común que los equipos trabajaran durante largos períodos sobre ramas aisladas. Cuando finalmente se intentaba integrar el trabajo, aparecían conflictos complejos, errores de compilación y fallas funcionales difíciles de diagnosticar. Este fenómeno fue denominado por algunos autores como "Integration Hell".
- La integración continua reduce este problema mediante una estrategia basada en pequeñas integraciones frecuentes.

¿Qué es Continuous Integration?

- Cada modificación enviada al repositorio desencadena automáticamente una serie de actividades:
  - Obtención del código fuente.
  - Compilación del proyecto.
  - Ejecución de pruebas unitarias.
  - Ejecución de análisis estático.
  - Generación de reportes.
  - Notificación de resultados.
- El objetivo principal es detectar errores tan pronto como son introducidos, reduciendo significativamente el costo de corrección.

Ejemplo Práctico

- Supongamos un equipo de cinco desarrolladores trabajando sobre una API REST.

  Sin CI:

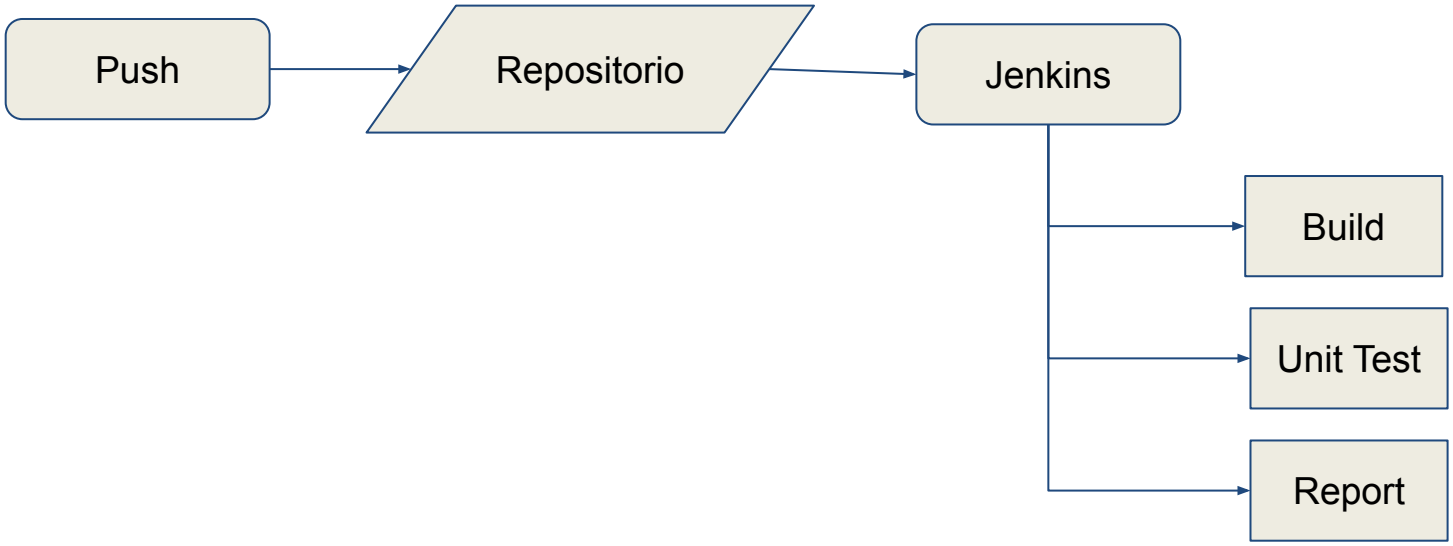
  - Cada desarrollador integra una vez por semana.
  - Los conflictos se descubren al final del sprint.
  - El tiempo promedio de resolución es elevado.

  Con CI:

  - Cada desarrollador integra varias veces al día.
  - Los conflictos aparecen inmediatamente.
  - El sistema proporciona retroalimentación en minutos.

Ejemplo Práctico

"Integrar frecuentemente reduce el riesgo acumulado."



Beneficios de Continuous Integration

- La integración continua no debe verse únicamente como una herramienta de automatización. Constituye una estrategia de reducción sistemática del riesgo técnico dentro del proceso de desarrollo.
- Uno de los beneficios más importantes es la detección temprana de defectos. Cuando una modificación genera un problema, el pipeline identifica inmediatamente qué cambio introdujo el error. Esto reduce drásticamente el tiempo necesario para localizar la causa raíz.
- Otro beneficio importante es la disminución de conflictos de integración. Las integraciones frecuentes implican que los desarrolladores trabajan con versiones recientes del sistema y sincronizan constantemente sus cambios con el resto del equipo.

Beneficios de Continuous Integration

- Además, CI favorece la calidad del software mediante la automatización de pruebas. Cada ejecución del pipeline verifica automáticamente cientos o miles de casos de prueba, proporcionando una validación mucho más rigurosa que las revisiones manuales.

- Desde la perspectiva organizacional, CI genera una cultura de responsabilidad compartida. Un build roto se convierte en un problema visible para todo el equipo y suele corregirse inmediatamente.

Arquitectura de un Pipeline CI/CD

- Un pipeline CI/CD puede interpretarse como una cadena automatizada de actividades que transforma código fuente en software desplegable.

- Los pipelines modernos suelen estar compuestos por varias etapas claramente diferenciadas.

Etapa 1: Source Control

- El repositorio actúa como fuente única de verdad.
- Todas las modificaciones deben pasar por él.

Arquitectura de un Pipeline CI/CD

Etapa 2: Build

- Se compila el código fuente y se generan artefactos ejecutables.
- Ejemplos:
  - JAR
  - WAR
  - Binarios
  - Imágenes Docker

Etapa 3: Testing

- Se ejecutan pruebas automatizadas:
  - Unitarias
  - Integración

Arquitectura de un Pipeline CI/CD

Etapa 4: Packaging

- Los artefactos son preparados para distribución.

Etapa 5: Deployment

- El software es desplegado automáticamente.

Integración de Herramientas en el Pipeline

- Un pipeline moderno no es una herramienta aislada. Es un ecosistema compuesto por múltiples aplicaciones especializadas que colaboran para automatizar el ciclo de vida del software.
- Una arquitectura típica puede incluir:
- Gestión de Código
  - Git
  - GitHub
  - GitLab
- Integración Continua
  - Jenkins

Integración de Herramientas en el Pipeline

- Testing
  - JUnit
  - Mockito
- Contenedores
  - Docker
- Gestión de Incidentes
  - Jira

- La integración entre estas herramientas permite construir una cadena continua de trazabilidad desde el requerimiento inicial hasta el despliegue en producción.

- Por ejemplo, una incidencia registrada en Jira puede asociarse a un commit, el commit puede activar un pipeline en Jenkins y dicho pipeline puede generar una nueva versión desplegada automáticamente.

Gestión de Incidencias dentro del Ciclo CI/CD

- En proyectos profesionales, el desarrollo de software no comienza con código sino con la identificación de necesidades, defectos o mejoras. Estas actividades suelen registrarse mediante sistemas de gestión de incidencias.
- Una incidencia puede representar:
  - Un error (bug).
  - Una nueva funcionalidad.
  - Una mejora de rendimiento.
  - Un requisito de seguridad.
  - Una tarea técnica.
- La integración entre herramientas de gestión de incidencias y plataformas CI/CD permite mantener una trazabilidad completa entre los requerimientos y las modificaciones realizadas sobre el sistema.
- Un flujo típico comienza cuando un usuario reporta un problema. El incidente es registrado, asignado a un desarrollador, corregido mediante una modificación de código y finalmente validado a través de un pipeline automatizado.

Gestión de Incidencias dentro del Ciclo CI/CD

- Este proceso permite responder preguntas fundamentales como:
  - ¿Qué cambio corrigió un determinado error?
  - ¿Quién implementó la solución?
  - ¿Qué pruebas validaron la corrección?
  - ¿En qué versión fue desplegada?
- La capacidad de responder estas preguntas resulta esencial en proyectos con altos requerimientos de calidad, auditoría o cumplimiento normativo.

Trazabilidad de Extremo a Extremo

- La trazabilidad consiste en la capacidad de relacionar todos los artefactos generados durante el ciclo de vida del software.
- La trazabilidad constituye una práctica esencial para sistemas complejos porque permite comprender cómo evolucionan los requisitos y cómo estos impactan sobre el diseño y la implementación.
- En un entorno CI/CD moderno la trazabilidad puede establecerse entre:
  - Requerimientos.
  - Historias de usuario.
  - Incidencias.
  - Commits.
  - Pull Requests.
  - Builds.
  - Casos de prueba.
  - Releases.
  - Despliegues.

Trazabilidad de Extremo a Extremo

- Por ejemplo, una historia de usuario registrada en Jira puede vincularse a una rama Git específica. Esa rama genera un Pull Request que activa un pipeline en Jenkins. El pipeline ejecuta pruebas y genera una nueva versión del producto.

- Toda esa información queda registrada y puede ser consultada posteriormente.

Jenkins: Introducción y Arquitectura General

- Jenkins es una plataforma de automatización de código abierto ampliamente utilizada para implementar prácticas de Integración Continua (CI) y Entrega Continua (CD). Su principal objetivo es automatizar tareas repetitivas relacionadas con la construcción, prueba y despliegue de software.
- Antes de herramientas como Jenkins, muchas organizaciones realizaban estas actividades manualmente, generando errores, inconsistencias y retrasos en la entrega.
- Jenkins permite transformar estas actividades en procesos repetibles, auditables y completamente automatizados.

Jenkins: Introducción y Arquitectura General

- Su popularidad se debe a varios factores:
  - Es software libre.
  - Posee una arquitectura extensible basada en plugins.
  - Puede integrarse con prácticamente cualquier herramienta del ecosistema DevOps.
  - Soporta múltiples lenguajes y tecnologías.
  - Permite definir pipelines como código.
- Actualmente Jenkins es utilizado en organizaciones de distintos tamaños, desde startups hasta grandes empresas tecnológicas.

Jenkins: Introducción y Arquitectura General

- Funciones principales
  - Compilación automática.
  - Ejecución de pruebas.
  - Generación de reportes.
  - Integración con repositorios Git.
  - Construcción de contenedores.
  - Despliegue automatizado.

Evolución de Jenkins dentro del Ecosistema DevOps

- Jenkins tiene su origen en el proyecto Hudson, desarrollado en 2004. Posteriormente la comunidad creó Jenkins, que evolucionó hasta convertirse en uno de los servidores de automatización más utilizados.
- Su crecimiento estuvo directamente relacionado con la adopción de prácticas DevOps.
- Inicialmente Jenkins era utilizado únicamente para:
  - Compilar proyectos.
  - Ejecutar pruebas.
- Con el tiempo incorporó capacidades más avanzadas:
- Primera generación
  - Automatización de builds.

Evolución de Jenkins dentro del Ecosistema DevOps

- Segunda generación
  - Integración continua completa.
- Tercera generación
  - Entrega continua.
- Cuarta generación
  - Integración con contenedores y cloud.
- Quinta generación
  - Pipeline as Code e Infrastructure as Code.
- Hoy Jenkins puede formar parte de arquitecturas modernas basadas en:
  - Docker.
  - Kubernetes.
  - Microservicios.

Instalación de Jenkins mediante Docker

- La utilización de contenedores simplifica significativamente la instalación de Jenkins.
- En lugar de instalar manualmente:
  - Java.
  - Dependencias.
  - Configuraciones.
- Podemos utilizar una imagen preconfigurada.
- Las ventajas son:
  - Reproducibilidad
  - Todos los entornos son idénticos.
  - Portabilidad
  - Puede ejecutarse en cualquier sistema compatible con Docker.
  - No afecta otras aplicaciones instaladas.
  - Facilidad de actualización
- Las nuevas versiones pueden desplegarse mediante nuevas imágenes.

Persistencia de Datos en Jenkins

- Uno de los errores más comunes al instalar Jenkins mediante Docker es no persistir los datos.

- Si el contenedor se elimina, también desaparecen:
  - Configuraciones.
  - Historial de builds.
  - Plugins.
  - Usuarios.

- Por esta razón es necesario utilizar volúmenes persistentes.

Gestión de Plugins en Jenkins

- La arquitectura de Jenkins se basa en plugins.
- La instalación básica proporciona únicamente funcionalidades mínimas.
- La mayoría de las capacidades avanzadas se incorporan mediante plugins.
- Ejemplos importantes:
  - Git Plugin
  - Pipeline Plugin
  - Docker Plugin
  - JUnit Plugin
  - Etc

Concepto de Job en Jenkins

- Un Job representa una tarea automatizada ejecutada por Jenkins.
- Puede incluir:
  - Compilación.
  - Testing.
  - Despliegue.
  - Generación de reportes.
- Tipos de Jobs:
  - Freestyle
    Configuración mediante interfaz gráfica.
  - Pipeline
    Configuración mediante código.

Concepto de Job en Jenkins

- Multibranch Pipeline
  - Soporte para múltiples ramas Git.
- Folder
  - Agrupación de proyectos.
- En entornos modernos se recomienda utilizar Pipeline Jobs debido a su trazabilidad y capacidad de versionado.

Pipeline as Code y Jenkinsfile

- Uno de los avances más importantes en Jenkins fue la introducción del concepto Pipeline as Code.
- La idea consiste en almacenar la definición completa del pipeline dentro del repositorio mediante un archivo llamado:
  - Jenkinsfile
- Esto permite:
  - Versionar pipelines.
  - Revisar cambios.
  - Replicar configuraciones.
  - Reducir errores manuales.
- El pipeline se convierte en parte del producto software.

Ejemplo

```
pipeline {
    agent any

    stages {

        stage('Build') {
            steps {
                echo 'Compilando'
            }
        }

        stage('Test') {
            steps {
                echo 'Ejecutando pruebas'
            }
        }

    }
}
```